

Nova CLI Guide

Command-Line Tools and NDI Disk Images

Nova Project

2026

Contents

1	Nova CLI User Guide	1
1.1	Quick Start	1
1.2	Mental Model	1
1.3	Build The CLI	2
1.4	Remote Host Syntax	2
1.5	File Types	3
1.6	Tested Examples	3
1.7	Local Image Commands	3
1.7.1	create	3
1.7.2	info	4
1.7.3	validate	4
1.7.4	label	4
1.7.5	dir	4
1.7.6	mkdir	4
1.7.7	rmdir	5
1.7.8	import	5
1.7.9	export	5
1.7.10	delete	6
1.8	BASIC Token Commands	6
1.8.1	tokenize	6
1.8.2	detokenize	6
1.9	Remote Device Commands	6
1.9.1	device status	6
1.9.2	device reboot	7
1.10	Drive Slot Commands	7
1.10.1	drive list	7
1.10.2	drive mount	7
1.10.3	drive unmount	7
1.11	WiFi Commands	8
1.11.1	wifi status	8
1.11.2	wifi scan	8
1.11.3	wifi set	8
1.11.4	WiFi Actions	8
1.12	Audio Commands	8
1.13	Remote Keyboard Command	9
1.14	Raw Remote SD Commands	9
1.14.1	ls	9
1.14.2	put	9
1.14.3	get	10
1.14.4	rm	10
1.15	Managed Remote Commands	10
1.15.1	Disk Images	10
1.15.2	ROMs	11

1.15.3	Soundfonts	11
1.15.4	Music	11
1.15.5	Assets	12
1.16	Standard SD Layout	12
1.17	Hardware Runtime Commands	12
1.18	Workflows	13
1.18.1	Create And Boot A Floppy	13
1.18.2	Unmount A Floppy And Fall Back To Default Boot	13
1.18.3	Create A Hard Disk Image	13
1.18.4	Round-Trip A BASIC Program	13
1.18.5	Upload Music And A Soundfont	13
1.19	Troubleshooting	13
1.20	Current Gaps	14
2	Nova Disk Image Format	15
2.1	Current Version	15
2.2	Image Layout	15
2.3	Header	15
2.4	BAM	16
2.5	Directory Entries	16
2.5.1	Entry Flags	16
2.5.2	File Types	17
2.6	Filenames	17
2.7	Directories	17
2.8	File Allocation	17
2.9	BASIC Files	18
2.10	Floppy And Hard Disk Images	18
2.11	Music Files	18
2.12	Validation	19
2.13	Known Format Limits	19

1 Nova CLI User Guide

`nova` is the command-line tool for Nova disk images and NovaHost SD-card management.

NDI is the disk image format. Nova is the CLI.

This guide is written for someone trying to get useful work done without reverse-engineering the code. It covers the implemented commands, their parameters, the paths they touch, and examples that are exercised by `tools/test-nova-cli-doc-examples.py`.

1.1 Quick Start

From the repository, run the managed CLI like this:

```
dotnet run --project e6502.Nova --
```

For shorter commands during development:

```
alias nova='dotnet run --project /Users/barry/Git/e6502/e6502.Nova --'
```

If you have published the NativeAOT binary, use it directly:

```
artifacts/nova-cli/osx-arm64/nova --help
```

Create a floppy-sized image, add a tokenized boot program, inspect it, and validate it:

```
nova create doc-demo.ndi --label DOCDEMO
nova import doc-demo.ndi AUTOBOOT.bas --tokenize
nova dir doc-demo.ndi
nova validate doc-demo.ndi
```

Upload that image to NovaHost as a floppy image:

```
nova disk upload doc-demo.ndi --remote 192.168.1.65 --floppy --name doc-demo.ndi
```

Mount it into fd0:

```
nova drive mount fd0 /disks/floppy/doc-demo.ndi --remote 192.168.1.65
```

Check the board:

```
nova device status --remote 192.168.1.65
```

1.2 Mental Model

Nova has three separate layers:

Layer	What It Is	Managed By
nova CLI	Host-side tool for images and NovaHost files	Your development machine
NDI	Disk image format for floppies and hard drives	e6502.Storage and NovaHost
NovaHost	ESP-side TCP command server and SD-card/device manager	The board

Floppy and hard-drive images use the same NDI format. The difference is size, where the image lives on the SD card, and which NovaHost slot mounts it.

NovaHost drive slots are:

```
fd0 fd1 fd2 fd3 hd0 hd1
```

Boot selection checks those slots in this order:

```
fd0, fd1, fd2, fd3, hd0, hd1
```

For each mounted slot, NovaHost looks for `AUTOBOOT.bas` or `AUTOBOOT.bin` in the image root. If no mounted image has an autoboot file, the first mounted floppy wins, then the first mounted hard drive.

Drive mount state is stored on the SD card in:

```
/config/boot.json
```

The CLI uploads images and files, manages mounted drive slots, updates WiFi settings, stops NovaHost-driven audio, reboots NovaHost, and can forward local terminal keypresses to the running NovaVM over WiFi. NovaHost speaks a newline-delimited JSON command protocol over TCP; the CLI is the supported way to drive it, and there is no HTTP/REST interface.

1.3 Build The CLI

Development build:

```
dotnet build e6502.Nova/e6502.Nova.csproj -c Release
```

`dotnet build` is for fast local compilation. `dotnet publish` for the Nova CLI defaults to NativeAOT and produces the binary users should run.

Publish a single NativeAOT binary for the current machine:

```
tools/publish-nova-cli.sh
```

Publish for a specific runtime:

```
tools/publish-nova-cli.sh osx-arm64
tools/publish-nova-cli.sh linux-x64
tools/publish-nova-cli.sh linux-arm64
tools/publish-nova-cli.sh win-x64
```

NativeAOT output paths:

```
artifacts/nova-cli/osx-arm64/nova
artifacts/nova-cli/linux-x64/nova
artifacts/nova-cli/linux-arm64/nova
artifacts/nova-cli/win-x64/nova.exe
```

The NativeAOT publish may report trim warnings for DryWetMidi. MIDI import and upload are still expected to work; verify with a `.mid` to `.nms` import/upload.

1.4 Remote Host Syntax

Remote commands talk to NovaHost over TCP.

`--remote` may appear before or after the command:

```
nova --remote 192.168.1.65 disk list
nova disk list --remote 192.168.1.65
nova disk list --remote=192.168.1.65
```

The host may include a port when testing against a local server:

```
nova device status --remote 127.0.0.1:8080
```

Remote paths are relative to the NovaHost SD-card root. These are equivalent:

```
nova get --remote 192.168.1.65 /config/boot.json
nova get --remote 192.168.1.65 config/boot.json
```

Backslashes are normalized to /. Leading slashes are stripped before building the SD-card path.

1.5 File Types

The CLI infers local NDI file type from extension:

Extension	NDI Type	Notes
.bas	BAS	Tokenized when imported with <code>--tokenize</code>
.sid	SID	SID music file
.bin	BIN	Binary program/data
.mid, .midi	MID	Compiled to .nms first
.nms	MID	Nova Music Stream
.nvg	GFX	Nova graphics asset
unknown	BIN	Stored as binary data

GFX is the internal NDI file type name for graphics. User-facing graphics assets should use `.nvg`.

1.6 Tested Examples

The examples in this guide are intended to be executable. Run the local image and remote-command documentation smoke test with:

```
tools/test-nova-cli-doc-examples.py
```

The test creates temporary NDI images, imports BASIC, binary, MIDI, and NVG files, validates image state, and runs remote commands against a local NovaHost-compatible mock server.

Remote examples using `192.168.1.65` are the same command shapes. Replace that host with your board's IP address.

1.7 Local Image Commands

Local image commands operate on `.ndi` files on the development machine.

1.7.1 create

Create and format an NDI image. Existing files are overwritten.

```
nova create <file.ndi> [--size <KB>|--hd] [--label <name>]
```

Options:

Option	Meaning
<code>--size <KB></code>	Image size in kilobytes. Default is 800.
<code>--hd</code>	Create a 64 MB image, equivalent to <code>--size 65536</code> .
<code>--label <name></code>	Volume label. Default is DISK.

Tested examples:

```
nova create doc-demo.ndi --label DOCDEMO
nova create hd0.ndi --hd --label HOME
nova create scratch.ndi --size 4096 --label SCRATCH
```

1.7.2 info

Print image metadata.

```
nova info <file.ndi>
```

Shows format version, volume label, sector size, total sectors, free sectors, directory location, data start, and directory capacity.

Tested example:

```
nova info doc-demo.ndi
```

1.7.3 validate

Check image consistency.

```
nova validate <file.ndi>
```

Validation checks that file allocations stay inside the image, data sectors are not double allocated, and BAM used/free counts match directory usage.

Tested example:

```
nova validate doc-demo.ndi
```

1.7.4 label

Update the image volume label in place.

```
nova label <file.ndi> <name>
```

The label must be 32 characters or fewer.

Tested example:

```
nova label doc-demo.ndi DOCS
```

1.7.5 dir

List a directory inside an image.

```
nova dir <file.ndi> [/path]
```

If /path is omitted, the root directory is listed.

Tested examples:

```
nova dir doc-demo.ndi
nova dir doc-demo.ndi /programs
```

1.7.6 mkdir

Create a directory inside an image.

```
nova mkdir <file.ndi> <path>
```

Parent directories must already exist.

Tested example:

```
nova mkdir doc-demo.ndi /programs
```

1.7.7 rmdir

Remove an empty directory from an image.

```
nova rmdir <file.ndi> <path>
```

Tested example:

```
nova mkdir doc-demo.ndi /empty
```

```
nova rmdir doc-demo.ndi /empty
```

1.7.8 import

Import a host file into an image.

```
nova import <file.ndi> <hostfile> [/dest] [--tokenize] [--tokens <path>]
```

Parameters and options:

Parameter	Meaning
<file.ndi>	Target image.
<hostfile>	Host file to import.
[/dest]	Destination directory inside the image. Defaults to /.
--tokenize	Tokenize ASCII NovaBASIC source before storing it.
--tokens <path>	Explicit tokenizer JSON file. Defaults to discovered ehbasic/tokens.json.

Behavior:

- Existing files with the same name in the destination directory are replaced.
- .mid and .midi files are compiled to .nms automatically.
- MIDI import cannot be combined with --tokenize.
- Tokenized BASIC import writes the normal two-byte BASIC load address prefix.
- Destination is only recognized when it starts with /.

Tested examples:

```
nova import doc-demo.ndi AUTOBOOT.bas /programs --tokenize
```

```
nova import doc-demo.ndi ehbasic/basic.bin
```

```
nova import doc-demo.ndi e6502.ESP32/novahost/assets/boot/novavm_logo.nvg /programs
```

```
nova import doc-demo.ndi docs/programs/midi/sousa-stars-stripes.mid /programs
```

The MIDI example stores `sousa-stars-stripes.nms` as type MID.

1.7.9 export

Export a file from an image.

```
nova export <file.ndi> <path> [hostdir-or-file] [--detokenize] [--tokens <path>]
```

Parameters and options:

Parameter	Meaning
<file.ndi>	Source image.
<path>	File path inside the image. Must name a file, not a directory.
[hostdir-or-file]	Existing directory or output filename. Defaults to current directory.
--detokenize	Convert tokenized BASIC to ASCII text.
--tokens <path>	Explicit tokenizer JSON file.

Tested examples:


```
nova export doc-demo.ndi /programs/AUTOBOOT.bas ./exported --detokenize
nova export doc-demo.ndi /basic.bin ./basic-export.bin
```

1.7.10 delete

Delete a file from an image.

```
nova delete <file.ndi> <path>
```

Tested example:

```
nova delete doc-demo.ndi /programs/novavm_logo.png
```

1.8 BASIC Token Commands

These commands operate on host files. They do not require an NDI image.

1.8.1 tokenize

Convert ASCII NovaBASIC source to tokenized BASIC with a two-byte load address prefix.

```
nova tokenize <input.txt> <output.bas> [--base <addr>] [--tokens <path>]
```

Options:

Option	Meaning
--base <addr>	BASIC load address, parsed as hexadecimal. Accepts \$0301, 0x0301, or 0301. Default is \$0301.
--tokens <path>	Explicit tokenizer JSON file.

If --tokens is omitted, the CLI searches upward from the executable directory and current working directory for ehbasic/tokens.json.

Tested example:

```
nova tokenize AUTOBOOT.bas AUTOBOOT.tokenized.bas --base $0301
```

1.8.2 detokenize

Convert a tokenized BASIC file back to ASCII text.

```
nova detokenize <input.bas> [output.txt] [--tokens <path>]
```

If no output path is supplied, text is written to stdout.

Tested examples:

```
nova detokenize AUTOBOOT.tokenized.bas
nova detokenize AUTOBOOT.tokenized.bas AUTOBOOT.roundtrip.txt
```

1.9 Remote Device Commands

1.9.1 device status

Read NovaHost health and SD-card status.

```
nova device status --remote <host>
```

This queries NovaHost's health and SD-card mount status over the TCP command protocol.

Tested example:

```
nova device status --remote 192.168.1.65
```

Use this first when the board is behaving oddly.

1.9.2 device reboot

Reboot NovaHost.

```
nova device reboot --remote <host>
```

Tested example:

```
nova device reboot --remote 192.168.1.65
```

This reboots the ESP-side NovaHost. It is not the same thing as a 6502 warm start or cold start.

1.10 Drive Slot Commands

Drive commands manage the six mounted NDI image slots on NovaHost.

```
nova drive list --remote <host>
```

```
nova drive mount <fd0|fd1|fd2|fd3|hd0|hd1> <sd-path.ndi> --remote <host>
```

```
nova drive unmount <fd0|fd1|fd2|fd3|hd0|hd1> --remote <host>
```

drives is accepted as an alias for drive.

1.10.1 drive list

List mounted and configured drive slots.

```
nova drive list --remote <host>
```

Tested example:

```
nova drive list --remote 192.168.1.65
```

1.10.2 drive mount

Mount an NDI image from the SD card into a drive slot and persist that mount in `/config/boot.json`.

```
nova drive mount <slot> <sd-path.ndi> --remote <host>
```

The SD path may be written with or without the leading `/`.

Tested examples:

```
nova drive mount fd0 /disks/floppy/doc-demo.ndi --remote 192.168.1.65
```

```
nova drive mount hd0 /disks/hard/hd0.ndi --remote 192.168.1.65
```

1.10.3 drive unmount

Unmount a drive slot and clear its persisted mount path.

```
nova drive unmount <slot> --remote <host>
```

Tested example:

```
nova drive unmount fd0 --remote 192.168.1.65
```

After unmounting, that image will not come back on the next boot unless it is mounted again.

1.11 WiFi Commands

WiFi commands manage NovaHost WiFi state and configuration.

```
nova wifi status --remote <host>
nova wifi scan --remote <host>
nova wifi set --remote <host> --ssid <ssid> [--password <password>] [--dhcp]
nova wifi set --remote <host> --ssid <ssid> [--password <password>] --static --static-ip <ip> --gateway <gateway>
nova wifi connect|disconnect|reconnect|forget --remote <host>
```

1.11.1 wifi status

Read configured and live WiFi state.

```
nova wifi status --remote 192.168.1.65
```

1.11.2 wifi scan

List nearby WiFi networks visible to NovaHost.

```
nova wifi scan --remote 192.168.1.65
```

1.11.3 wifi set

Update WiFi configuration.

DHCP example:

```
nova wifi set --remote 192.168.1.65 --ssid NovaLab --password secret --dhcp
```

Static IP example:

```
nova wifi set --remote 192.168.1.65 --ssid NovaLab --password secret \
  --static --static-ip 192.168.1.65 --gateway 192.168.1.1 \
  --subnet 255.255.255.0 --dns 192.168.1.1
```

Aliases:

Option	Aliases
--password	--pass
--static-ip	--ip
--gateway	--gw
--subnet	--netmask

1.11.4 WiFi Actions

```
nova wifi connect --remote 192.168.1.65
nova wifi disconnect --remote 192.168.1.65
nova wifi reconnect --remote 192.168.1.65
nova wifi forget --remote 192.168.1.65
```

forget clears the stored WiFi configuration.

1.12 Audio Commands

Audio commands manage NovaHost-driven audio playback.

```
nova audio status --remote <host>
nova audio stop --remote <host>
```

Tested examples:

```
nova audio status --remote 192.168.1.65
nova audio stop --remote 192.168.1.65
```

1.13 Remote Keyboard Command

`nova keyboard` forwards keypresses from the local terminal to the running NovaVM through NovaHost's debug TCP service. This is a WiFi keyboard stopgap for hardware sessions where the board-side USB keyboard path is not usable.

```
nova keyboard --remote <host> [--port 6503] [--echo] [--ctrl-c-quits]
```

Example:

```
nova keyboard --remote 192.168.1.65
```

Control keys:

- `Ctrl-]` exits the local bridge.
- `Ctrl-C` is sent to Nova as byte `$03`; pass `--ctrl-c-quits` to make it exit the bridge instead.
- Escape and F1 also send Nova's `$03` break byte.
- Enter, Backspace, Tab, Delete, printable ASCII, and arrow keys are forwarded. Arrow keys use the same `$1C` through `$1F` bytes as the desktop emulator; the active Nova editor or program decides what those bytes mean.

This command uses TCP port 6503, not NovaHost's standard command port. Use `--port` only when the debug service is exposed on a non-default port.

1.14 Raw Remote SD Commands

Raw remote commands map directly to NovaHost SD-card paths.

```
nova ls --remote <host> [path]
nova put --remote <host> <local-path> [remote-path]
nova get --remote <host> <remote-path> [local-path]
nova rm --remote <host> <remote-path>
```

1.14.1 ls

List an SD directory.

```
nova ls --remote <host> [path]
```

Tested examples:

```
nova ls --remote 192.168.1.65
nova ls --remote 192.168.1.65 disks/floppy
```

1.14.2 put

Upload a local file to an SD path.

```
nova put --remote <host> <local-path> [remote-path]
```

If the local file is `.mid` or `.midi`, `put` compiles it to `.nms` before upload. For all other extensions, bytes are uploaded exactly as supplied.

Tested examples:

```
nova put --remote 192.168.1.65 doc-demo.ndi disks/floppy/doc-demo.ndi
nova put --remote 192.168.1.65 docs/programs/midi/sousa-stars-stripes.mid music/STARS.NMS
```

1.14.3 get

Download an SD file.

```
nova get --remote <host> <remote-path> [local-path]
```

Tested example:

```
nova get --remote 192.168.1.65 config/boot.json ./boot.json
```

1.14.4 rm

Delete an SD file or empty directory.

```
nova rm --remote <host> <remote-path>
```

Tested example:

```
nova rm --remote 192.168.1.65 tmp/old.bin
```

NovaHost rejects writes/deletes against mounted .ndi files. Unmount the slot first.

1.15 Managed Remote Commands

Managed remote commands place files under standard SD-card directories.

1.15.1 Disk Images

```
nova disk list --remote <host> [--hard|--hd|--floppy|--fd] [--path <path>]
```

```
nova disk upload <file.ndi> --remote <host> [--hard|--hd|--floppy|--fd] [--name <name>] [--path <path>]
```

```
nova disk download <name-or-path> --remote <host> [local-path] [--hard|--hd|--floppy|--fd] [--path <path>]
```

```
nova disk delete <name-or-path> --remote <host> [--hard|--hd|--floppy|--fd] [--path <path>]
```

Standard directories:

Image Kind	SD Directory
Floppy	/disks/floppy
Hard disk	/disks/hard

Placement rules for upload:

- `--floppy` or `--fd` forces `/disks/floppy`.
- `--hard` or `--hd` forces `/disks/hard`.
- `--path <path>` is an explicit SD path and overrides disk type inference.
- Without a flag, filenames beginning with `hd` go to `/disks/hard`.
- Without a flag, images at least 16 MB go to `/disks/hard`.
- Everything else defaults to `/disks/floppy`.

Tested examples:

```
nova disk list --remote 192.168.1.65
```

```
nova disk list --remote 192.168.1.65 --floppy
```

```
nova disk upload doc-demo.ndi --remote 192.168.1.65 --floppy --name doc-demo.ndi
```

```
nova disk download doc-demo.ndi --remote 192.168.1.65 --floppy ./doc-demo.downloaded.ndi
```

```
nova disk delete doc-demo.ndi --remote 192.168.1.65 --floppy
```

Uploading a disk image only copies it to the SD card. It does not mount the image into `fd0`, `fd1`, `hd0`, etc.

1.15.2 ROMs

ROM commands place files under `/roms`.

```
nova rom list --remote <host>
nova rom upload <file> --remote <host> [--name <name>] [--path <path>]
nova rom download <name-or-path> --remote <host> [local-path] [--path <path>]
nova rom delete <name-or-path> --remote <host> [--path <path>]
```

If `--name` has no extension, the local file extension is appended.

Tested examples:

```
nova rom upload ehbasic/basic.bin --remote 192.168.1.65 --name novabasic.bin
nova rom list --remote 192.168.1.65
nova rom download novabasic.bin --remote 192.168.1.65 ./novabasic.downloaded.bin
```

Uploading ROM files does not automatically reload them into the running FPGA. Use the existing host-control tooling for runtime reloads.

1.15.3 Soundfonts

Soundfont commands place files under `/soundfonts`.

```
nova soundfont list --remote <host>
nova soundfont upload <file.nsf> --remote <host> [--name <name>] [--path <path>]
nova soundfont download <name-or-path> --remote <host> [local-path] [--path <path>]
nova soundfont delete <name-or-path> --remote <host> [--path <path>]
```

Current limitation: upload accepts only Nova-native `.nsf` banks. Uploading an `.sf2` is rejected because SF2-to-NSF conversion is not implemented in this CLI.

Tested examples:

```
nova soundfont upload DocBank.nsf --remote 192.168.1.65
nova soundfont list --remote 192.168.1.65
nova soundfont download DocBank.nsf --remote 192.168.1.65 ./DocBank.downloaded.nsf
```

1.15.4 Music

Music commands place files under `/music`.

```
nova music list --remote <host>
nova music upload <file.mid|file.midi|file.nms> --remote <host> [--name <name>] [--path <path>]
nova music download <name-or-path> --remote <host> [local-path] [--path <path>]
nova music delete <name-or-path> --remote <host> [--path <path>]
```

Behavior:

- `.mid` and `.midi` are compiled to `.nms` at upload time.
- `.nms` is uploaded directly.
- The uploaded song is not tied to a soundfont.
- MIDPLAY uses the loaded soundfont, or asks NovaHost to load the default soundfont if one is available and none is already loaded.

Tested examples:

```
nova music upload docs/programs/midi/sousa-stars-stripes.mid --remote 192.168.1.65 --name STARS.NMS
nova music list --remote 192.168.1.65
nova music download STARS.NMS --remote 192.168.1.65 ./STARS.downloaded.NMS
```

MIDI compiler notes:

- SMPTE MIDI timing is not supported.

- The compiler emits Nova Music Stream version 2 (NMS2).
- Newly compiled streams preserve MIDI title, author, and copyright metadata when those MIDI meta events are present.
- Initial tempo defaults to 120 BPM until tempo events change it.
- Events are routed to up to 8 WTS voices.
- Program, note, velocity, channel, and frame timestamp are preserved.

1.15.5 Assets

Asset commands place files under `/assets` or a typed asset subdirectory.

```
nova asset list --remote <host> [--type|--kind|-t <type>]
nova asset upload <file> --remote <host> --type|--kind|-t <type> [--name <name>] [--path <path>]
nova asset download <name-or-path> --remote <host> [--type|--kind|-t <type>] [local-path] [--path <path>]
nova asset delete <name-or-path> --remote <host> [--type|--kind|-t <type>] [--path <path>]
```

Type mapping:

Type	SD Directory
boot	/assets/boot
font, fonts	/assets/fonts
sid, sid-assets, sid_assets	/assets/sid
music, mid, midi	/music
any other type	/assets/<type>

Tested examples:

```
nova asset upload e6502.ESP32/novahost/assets/boot/novavm_logo.nvg \
  --remote 192.168.1.65 --type boot

nova asset list --remote 192.168.1.65 --type boot
nova asset download novavm_logo.nvg --remote 192.168.1.65 --type boot ./novavm_logo.downloaded.nvg
```

1.16 Standard SD Layout

NovaHost currently uses this SD-card layout:

```
/config/boot.json
/roms/novabasic.bin
/roms/extension.bin
/disks/floppy/*.ndi
/disks/hard/*.ndi
/soundfonts/*.nsfb
/music/*.nms
/assets/boot/*.nvg
/assets/fonts/*
/assets/sid/*
```

1.17 Hardware Runtime Commands

For normal hardware work, use the first-class `nova` commands:

```
nova device status --remote 192.168.1.65
nova drive list --remote 192.168.1.65
nova drive mount fd0 /disks/floppy/doc-demo.ndi --remote 192.168.1.65
nova drive unmount fd0 --remote 192.168.1.65
```

```
nova wifi status --remote 192.168.1.65
nova wifi scan --remote 192.168.1.65
nova wifi set --remote 192.168.1.65 --ssid NovaLab --password secret --dhcp
nova audio status --remote 192.168.1.65
nova audio stop --remote 192.168.1.65
nova keyboard --remote 192.168.1.65
nova device reboot --remote 192.168.1.65
```

The raw NovaHost SD commands are intentionally not the primary user-facing workflow. The CLI still exposes raw SD file operations as `nova ls`, `nova put`, `nova get`, and `nova rm` for compatibility and diagnostics.

1.18 Workflows

1.18.1 Create And Boot A Floppy

```
nova create doc-demo.ndi --label DOCDEMO
nova import doc-demo.ndi AUTOBOOT.bas --tokenize
nova validate doc-demo.ndi
nova disk upload doc-demo.ndi --remote 192.168.1.65 --floppy --name doc-demo.ndi
nova drive mount fd0 /disks/floppy/doc-demo.ndi --remote 192.168.1.65
```

After a cold start, `fd0` has first boot priority.

1.18.2 Unmount A Floppy And Fall Back To Default Boot

```
nova drive unmount fd0 --remote 192.168.1.65
nova device reboot --remote 192.168.1.65
```

Because unmount clears the persisted path, the image will not come back on the next boot unless it is mounted again.

1.18.3 Create A Hard Disk Image

```
nova create hd0.ndi --hd --label HOME
nova disk upload hd0.ndi --remote 192.168.1.65 --hard --name hd0.ndi
nova drive mount hd0 /disks/hard/hd0.ndi --remote 192.168.1.65
```

1.18.4 Round-Trip A BASIC Program

```
nova export doc-demo.ndi /AUTOBOOT.bas ./AUTOBOOT.txt --detokenize
# edit AUTOBOOT.txt
nova import doc-demo.ndi AUTOBOOT.txt / --tokenize
nova validate doc-demo.ndi
```

1.18.5 Upload Music And A Soundfont

```
nova soundfont upload DocBank.nsfb --remote 192.168.1.65
nova music upload docs/programs/midi/sousa-stars-stripes.mid \
  --remote 192.168.1.65 --name STARS.NMS
```

On NovaBASIC, use the music commands (`SFLOAD`, `MIDPLAY`, `MIDSTOP`) to load and play these files.

1.19 Troubleshooting

`tokens.json` not found

: Run from the repository or pass `--tokens ehbasic/tokens.json`.

Cannot combine `--tokenize` with MIDI music compilation

: Import BASIC and MIDI as separate files. MIDI import always compiles to `.nms`.

SF2 upload must convert to Nova-native `.nsfb` first

: Upload an existing `.nsfb` bank. The SF2 converter is not implemented in this CLI.

PUT ... 409 Conflict

: NovaHost rejects writes to mounted `.ndi` files. Unmount the drive first.

GET ... 404 Not Found

: Check whether the path is relative to the SD-card root, and whether a managed command is prepending a base directory.

503 sd not mounted

: NovaHost does not see the SD card. Check `nova device status`, power, card seating, and FAT32 formatting.

Not enough contiguous free sectors

: NDI images allocate contiguous sectors. Delete/reimport churn can fragment an image. Create a fresh image or export/rebuild it.

Directory is full

: The image has a fixed directory table. Remove files/directories or extend the image format.

1.20 Current Gaps

The implemented CLI does not yet have first-class verbs for:

- reloading ROMs after upload
- converting `.sf2` soundfonts to `.nsfb`

ROM reload is still handled by existing host-control tooling. Soundfont conversion should become part of the CLI when the `.sf2` to `.nsfb` converter is implemented.

2 Nova Disk Image Format

This document describes the current Nova Disk Image (`.ndi`) format implemented by `e6502.Storage`.

NDI is a simple, seekable disk image format used by Nova storage devices. It is not a FAT filesystem. It is intentionally small enough for NovaHost and the 6502-side file I/O path to reason about directly.

2.1 Current Version

Field	Value
Magic	N, D, I, \$1A
Format version	2
Sector size	256 bytes
Byte order	little-endian
Header size	256 bytes
Directory size	48 sectors
Directory entry size	64 bytes
Directory capacity	192 entries
Root parent index	\$FFFF

2.2 Image Layout

The image is laid out as:

sector 0	header
sector 1.. <code>dirStart</code> -1	BAM sectors
<code>dirStart</code> .. <code>dataStart</code> -1	directory sectors
<code>dataStart</code> .. <code>end</code>	file data sectors

The data sector numbers stored in directory entries are relative to the data area, not physical image sectors.

Physical byte offset for a file data sector is:

`(header.DataStartSector + entry.StartSector) * header.SectorSize`

2.3 Header

The header is one 256-byte sector at the start of the image.

Offset	Size	Description
\$00	4	magic bytes: 4E 44 49 1A (NDI + EOF marker)
\$04	1	format version, currently 2
\$05	2	sector size, currently 256
\$07	1	reserved, zero

Offset	Size	Description
\$08	4	total sectors in the image
\$0C	32	volume label, null-padded ASCII
\$2C	4	directory start sector
\$30	4	directory sector count
\$34	4	data start sector
\$38	4	free data sector count
\$3C	196	reserved, zero

The format version lives at \$04. Older notes sometimes assumed it lived at \$03, but \$03 is part of the magic sentinel.

2.4 BAM

The block allocation map tracks data sectors only.

Each bit represents one data sector:

Bit value	Meaning
0	free
1	allocated

Bit order within each byte is least-significant bit first:

```
sector 0 -> byte 0 bit 0
sector 1 -> byte 0 bit 1
...
sector 7 -> byte 0 bit 7
sector 8 -> byte 1 bit 0
```

The BAM byte stream is padded to occupy full 256-byte sectors between the header and directory.

2.5 Directory Entries

Each directory entry is 64 bytes. Four entries fit in one 256-byte sector.

Offset	Size	Description
\$00	1	flags
\$01	1	file type
\$02	2	parent directory entry index, \$FFFF for root
\$04	4	start sector, relative to data area
\$08	4	size in bytes
\$0C	32	filename, null-padded ASCII
\$2C	4	allocated sector count
\$30	16	reserved

2.5.1 Entry Flags

Bit	Mask	Meaning
0	\$01	active entry

Bit	Mask	Meaning
1	\$02	directory
7	\$80	locked

Inactive entries have flags byte 0 and may be reused.

2.5.2 File Types

Type	ID	Typical extensions
BAS	0	.bas
SID	1	.sid
BIN	2	.bin, .xram, unknown extensions
MID	3	.mid, .midi, .nms
GFX	4	.nvg
DIR	5	directories

GFX is the internal NDI file type name for graphics. User-facing graphics assets should use .nvg.

2.6 Filenames

Filenames are stored as ASCII, null-padded, maximum 32 bytes. The implementation truncates longer names.

Directory lookup is case-insensitive in current host code.

File names are stored with their extension, for example:

```
AUTOBOOT.bas
KEYBOARD.BIN
STARS.NMS
SPLASH.NVG
```

NovaBASIC display and load paths may strip or infer extensions depending on the runtime command being used, but the NDI directory stores the full filename.

2.7 Directories

Directories are represented as directory entries with:

- active flag set
- directory flag set
- file type DIR
- size 0
- start sector 0
- sector count 0

Children refer to the directory entry index in their parent field. Root entries use parent \$FFFF.

Directories do not have separate data sectors.

2.8 File Allocation

Files are allocated as contiguous runs of data sectors.

Current write behavior:

1. If a file with the same name exists in the target directory, it is deleted.
2. The BAM searches for the first contiguous run large enough for the new file.
3. File bytes are written to the data area.
4. The final sector is zero-padded.
5. A directory entry records byte size and allocated sector count.

This is simple and fast, but it means images can become fragmented after delete/reimport churn. A write can fail even if total free space is large when no contiguous run is large enough.

2.9 BASIC Files

Tokenized NovaBASIC files stored in NDI images should include a two-byte load address prefix. The normal BASIC load address is \$0301.

The Nova CLI's `--tokenize` flow writes this prefix automatically.

```
nova import demo.ndi AUTOBOOT.bas --tokenize
```

The standalone token command also writes the prefix:

```
nova tokenize AUTOBOOT.txt AUTOBOOT.bas --base $0301
```

2.10 Floppy And Hard Disk Images

Floppy and hard disk images use the same NDI on-disk format. The image size and NovaHost mount slot determine how the system treats the image.

The Nova CLI defaults to an 800 KB floppy-sized image:

```
nova create fd0.ndi --label BOOT
```

The CLI's `--hd` shortcut creates a 64 MB hard-disk-sized image:

```
nova create hd0.ndi --hd --label HOME
```

Arbitrary image sizes are also supported:

```
nova create scratch.ndi --size 4096 --label SCRATCH
```

Mounting is not stored inside the NDI image. NovaHost stores drive-slot mount configuration in `/config/boot.json` on the SD card.

NovaHost boot slot order is:

```
fd0, fd1, fd2, fd3, hd0, hd1
```

For each mounted slot, NovaHost looks for `AUTOBOOT.bas` or `AUTOBOOT.bin` in the image root. If no autoboot file is found, the first mounted floppy is the default device, followed by the first mounted hard drive.

2.11 Music Files

MIDI files imported or uploaded through the Nova CLI are compiled to Nova Music Stream (`.nms`) first. The resulting file is stored as file type MID.

The current `.nms` compiler emits NMS2 streams for the ESP/WTS playback path. The fixed header is little-endian and starts with:

Offset	Size	Description
\$00	4	Magic NMS2
\$04	2	Format version (2)
\$06	2	Header size in bytes

Offset	Size	Description
\$08	2	Event record size (10)
\$0A	2	WTS sample rate
\$0C	4	Event stream offset
\$10	4	Event stream byte count
\$14	4	Event record count
\$18	4	Total audio frames

When the header size is at least 128 bytes, offsets \$20, \$40, and \$60 contain 32-byte null-padded ASCII title, author, and copyright strings copied from MIDI metadata. Older 32-byte headers remain valid; players must use the event stream offset rather than assuming events begin immediately after the fixed header.

The `.nms` file is not coupled to a soundfont. Playback uses the currently loaded soundfont, or asks NovaHost to load a default soundfont if one is available.

2.12 Validation

`nova validate <file.ndi>` checks:

- file allocations stay within the image
- no data sector is claimed by multiple files
- BAM allocation counts match directory allocation counts

Validation does not currently verify every reserved byte or filename character.

2.13 Known Format Limits

- Fixed 192-entry directory table.
- Contiguous allocation only.
- No journaling.
- No timestamps.
- No long filename extension table.
- No sparse files.
- No per-file checksum.
- Host APIs currently read some file sizes into `int`, although the on-disk directory size field is 32-bit.

These limits are acceptable for the current Nova storage layer, but should be revisited if NDI becomes the long-term hard disk format.